

Open-Weight LLM Platform — Team Cookbook

Eleven field-tested recipes for standing up an enterprise-grade open-weight LLM inference platform for developer teams — on-premises, on Azure Kubernetes Service, or on Azure AI Foundry. Condensed from the full playbook; each recipe links back to its deep-dive chapter.

Source and updates: github.com/heisenberg-alt/oss-model-playbook · If this cookbook saves your team time, star the repository — it helps other teams find it.

RECIPES

1. Choose your deployment track
2. Size the cluster
3. Build the 3-tier model catalog
4. Serve with vLLM on Kubernetes
5. AKS quickstart
6. Foundry serverless quickstart
7. Stand up the gateway
8. Connect the IDEs
9. Benchmark and set SLOs
10. Know your cost break-evens
11. Security and governance baseline

RECIPE 1

Choose your deployment track

When to use: day zero. Everything downstream depends on this call.

QUESTION	IF YES
Must data stay on your premises (hard regulatory requirement)?	Track A — on-prem Kubernetes, vLLM + NVIDIA GPU Operator
Cloud acceptable and you want cluster-level control or custom models?	Track B — AKS with KAITO add-on or self-managed vLLM
Want zero cluster operations, spiky or low volume?	Track C1 — Azure AI Foundry serverless, pay-per-token
Want zero cluster operations, sustained high volume?	Track C2 — Foundry managed compute, dedicated GPU VMs

The hybrid pattern (most common at 50+ devs): dedicated GPUs (A/B) serve the default coder model at high utilization; the gateway routes the few percent of frontier-reasoning traffic to Foundry serverless pay-per-token. Best of both cost curves.

Full matrix and rationale: [playbook chapter 01 · Decision Framework](#).

RECIPE 2

Size the cluster

When to use: before buying or reserving any GPU.

TEAM SIZE	PEAK CONCURRENT REQUESTS	SUSTAINED THROUGHPUT	REFERENCE HARDWARE
20 devs	4–8 in flight	1,000–3,000 out-tok/s	2× L40S or 1× A100/H100 80 GB
50 devs	10–20 in flight	3,000–8,000 out-tok/s	2× A100/H100 + spot burst
100 devs	20–45 in flight	6,000–15,000 out-tok/s	4–8× A100/H100 across 2+ nodes

- **Heuristic:** one 80 GB-class GPU per 20–25 developers (mixed workloads, MoE default model); halve density for agent-heavy teams.
- **KV cache is the concurrency budget:** serving 64K context instead of 256K gives roughly 4× more concurrent sequences on the same GPU.
- **Pilot before purchase (mandatory for on-prem):** two weeks on cloud GPUs with 5–10 representative devs; capture tokens per request and peak concurrency from gateway logs; scale linearly and add 40% headroom.

Worked examples and KV-cache math: chapter 02 · Capacity Planning.

RECIPE 3

Build the 3-tier model catalog

When to use: always. One giant model for everything costs 3–5× more than routed tiers.

TIER	ROLE	RECOMMENDED MODEL	TRAFFIC SHARE
Fast	Completions, commit messages, quick Q&A	Qwen2.5-Coder-7B (AWQ/FP8)	30–50% of requests, ~10% of tokens
Default	Chat, code generation, agentic development	Qwen3-Coder-30B-A3B MoE (FP8)	40–60% of requests, ~60% of tokens
Reasoning	Hard debugging, architecture analysis	gpt-oss-120B / DeepSeek-R1-Distill-32B / Foundry serverless R1	5–10% of requests, ~30% of tokens

Frontier watch (July 2026): DeepSeek-V4 (Pro 1.6T / Flash 284B, 1M context, MIT), GLM-5.2 (753B, 1M context, MIT) and Kimi K2.7-Code (1T, Modified MIT) lead the open frontier; Kimi K3 (2.8T, 1M context) releases open weights 2026-07-27 — until then it is hosted-API only. Trial frontier models via serverless APIs before committing cluster capacity.

Full catalog with hardware footprints: the interactive model picker in the repository.

RECIPE 4

Serve with vLLM on Kubernetes

When to use: Tracks A and B2. Ollama is a workstation tool; 20+ concurrent developers need vLLM-class continuous batching.

```
args:
  - --model=/models/qwen3-coder-30b-fp8
  - --served-model-name=qwen3-coder-30b
  - --max-model-len=65536          # context cap = concurrency budget
  - --max-num-seqs=32
  - --gpu-memory-utilization=0.92
  - --enable-prefix-caching        # agents re-send context; big TTFT win
  - --enable-chunked-prefill       # protects TPOT from long-prompt stalls
  - --enable-auto-tool-choice
  - --tool-call-parser=qwen3_coder
```

- Weights on local NVMe (/var/lib/models); startup probe with a generous failure threshold — model load takes minutes.
- MIG-slice large GPUs for the fast tier; tensor parallelism only across NVLink-connected GPUs.
- Autoscale on `vllm:num_requests_waiting` (queue depth), never on CPU; scale down slowly (600 s window).

Complete manifests, GPU Operator setup, and air-gap notes: [chapter 03 · On-Prem Kubernetes](#).

RECIPE 5

AKS quickstart

When to use: Track B. Start with the KAITO add-on for speed; move hot paths to self-managed vLLM when you need tuning surface.

```
az aks create -g $RG -n $CLUSTER -l $LOC \  
  --tier standard --node-count 2 --node-vm-size Standard_D8ds_v5 \  
  --enable-managed-identity --enable-oidc-issuer --enable-workload-identity \  
  --enable-azure-monitor-metrics \  
  --enable-ai-toolchain-operator \  
  --generate-ssh-keys
```

- Request GPU quota (NCads A100 v4 / NCads H100 v5) before anything else — approval can take days.
- Base pool reserved (1–3 yr, 30–60% off PAYG); burst pool on spot with eviction-tolerant replicas only.
- Deleting a KAITO workspace does not delete its GPU node pool — delete it explicitly to stop billing.

KAITO workspaces, node pools, KEDA autoscaling, Azure-native security: chapter 04 · AKS.

RECIPE 6

Foundry serverless quickstart

When to use: Track C1, the reasoning tier for any track, and trialing new models before pulling weights into the cluster.

```
az cognitiveservices account create -n llm-foundry -g $RG -l $LOC \
  --kind AIServices --sku S0

az cognitiveservices account deployment create -n llm-foundry -g $RG \
  --deployment-name deepseek-r1 \
  --model-name DeepSeek-R1 --model-format DeepSeek --model-version "1" \
  --sku-name GlobalStandard --sku-capacity 1
```

- Prefer Entra ID (keyless) auth; pick regional processing if you have residency constraints.
- Serverless usually wins below ~30–60M tokens/day team-wide; above that, dedicated compute wins.
- Managed compute (C2) has no built-in content filtering — put guardrails in the gateway.

Managed compute deployment and blue/green rollouts: chapter 05 · Foundry.

RECIPE 7

Stand up the gateway

When to use: all tracks, day one. Never expose raw model endpoints to developers.

```
curl -s http://litellm.llm.svc:4000/key/generate \
-H "Authorization: Bearer $MASTER_KEY" -H "Content-Type: application/json" \
-d '{
  "team_id": "payments-squad",
  "models": ["fast", "default"],
  "max_budget": 50, "budget_duration": "1d",
  "rpm_limit": 300, "tpm_limit": 500000
}'
```

- **Model aliasing is the superpower:** devs call `fast` / `default` / `reasoning`; upgrading the underlying model is a gateway config change, not a 100-dev migration.
- Every team gets `fast` + `default`; `reasoning` is allow-listed (it is the expensive tier).
- Fallback chains: `default` → `reasoning` on saturation, `fast` → `default` on failure; spot evictions become invisible.
- Request/response content logging OFF; metadata (tokens, latency, model, team) only.

Full LiteLLM deployment and routing config: chapter 06 · Gateway & DevEx.

RECIPE 8

Connect the IDEs

When to use: onboarding. One base URL for everything:

https://llm.internal.example.com/v1.

CLIENT	INTEGRATION
VS Code — Copilot BYO model	Manage Models → custom OpenAI-compatible provider → gateway URL + team key; expose <code>default</code> and <code>reasoning</code>
Continue / Cline / Roo	<code>provider: openai</code> , <code>apiBase</code> = gateway URL, <code>model</code> = <code>default</code>
Aider	<code>OPENAI_API_BASE</code> + <code>OPENAI_API_KEY</code> env vars, <code>--model</code> <code>openai/default</code>
Tab completion (FIM)	Point at the <code>fast</code> tier only — latency-critical, high QPS
Offline laptop	<code>ollama run qwen3-coder:30b</code> locally; the gateway remains the shared source of truth

RECIPE 9

Benchmark and set SLOs

When to use: before launch, after every engine upgrade, model swap, or flag change.

SLO	TARGET
Chat TTFT p95	< 1.5 s (page at > 3 s for 10 min)
Completions TTFT p95	< 400 ms
TPOT p95	< 60 ms ($\geq$ ~17 tok/s per stream)
Queue time p95	< 500 ms (capacity review at > 2 s daily peaks)
Preemption rate	~0 (> 1% = KV over-commit)

```
vllm bench serve --backend openai-chat \  
  --base-url https://llm.internal.example.com/v1 --model default \  
  --dataset-name random --random-input-len 8192 --random-output-len 512 \  
  --num-prompts 300 --request-rate 4      # sweep 1..16, find the SLO knee
```

Goodput — the highest request rate where p95 TTFT and TPOT still meet SLO — is the only capacity number that matters. Tuning levers in ROI order: quantization (FP8/AWQ), prefix caching, chunked prefill, context caps, `max-num-seqs`. Chapter 07 · Performance.

RECIPE 10

Know your cost break-evens

RULE OF THUMB	NUMBER (MID-2026, RE-VERIFY)
Serverless vs one dedicated A100 (3-yr reserved)	Dedicated wins above ~20–50M output tokens/month on that GPU
On-prem vs AKS reserved	On-prem needs > ~50–60% sustained utilization plus existing DC and platform staffing
Reasoning tier at $\leq$ 10% traffic share	Serverless almost always beats owning 2× H100 for it
Effective H100 cost, on-prem 24×7	~\$1.3–1.9K/month vs ~\$5–8K/month cloud PAYG-equivalent

Own the base load, rent the peaks and the frontier. Re-quote serverless prices quarterly — per-token prices keep falling, so the break-even moves. Chapter 10 · Cost & TCO.

RECIPE 11

Security and governance baseline

- ☐ Gateway-only exposure; TLS everywhere; NetworkPolicy default-deny in the LLM namespace
- ☐ Virtual keys per team, auto-expiring, rotated quarterly; OIDC/Entra SSO where possible
- ☐ Prompts and completions processed in memory, never persisted; metadata-only logging
- ☐ Weights pulled from official Hugging Face orgs only; revisions pinned, checksums verified
- ☐ Engine pods have no internet egress after weights are cached (`HF_HUB_OFFLINE=1`)
- ☐ License review before catalog entry: Apache-2.0/MIT safe defaults; Llama/Gemma need legal sign-off; Kimi Modified MIT needs clause review
- ☐ Per-key RPM/TPM limits and budgets — a runaway agent loop is a governance event, not an outage
- ☐ Audit metadata (who, which model, when, token counts) retained ≥ 1 year

Threat model, data-handling policy, compliance map: chapter 09 · Security & Governance.

Verification note — model names, VM SKUs, prices, and operator versions were checked against public documentation as of July 2026. GPU pricing and model availability drift monthly; re-verify against primary sources before committing budget.

Full playbook, interactive model picker, and updates: github.com/heisenberg-alt/oss-model-playbook. If your team uses this cookbook, please star the repository — stars are how other teams discover it.